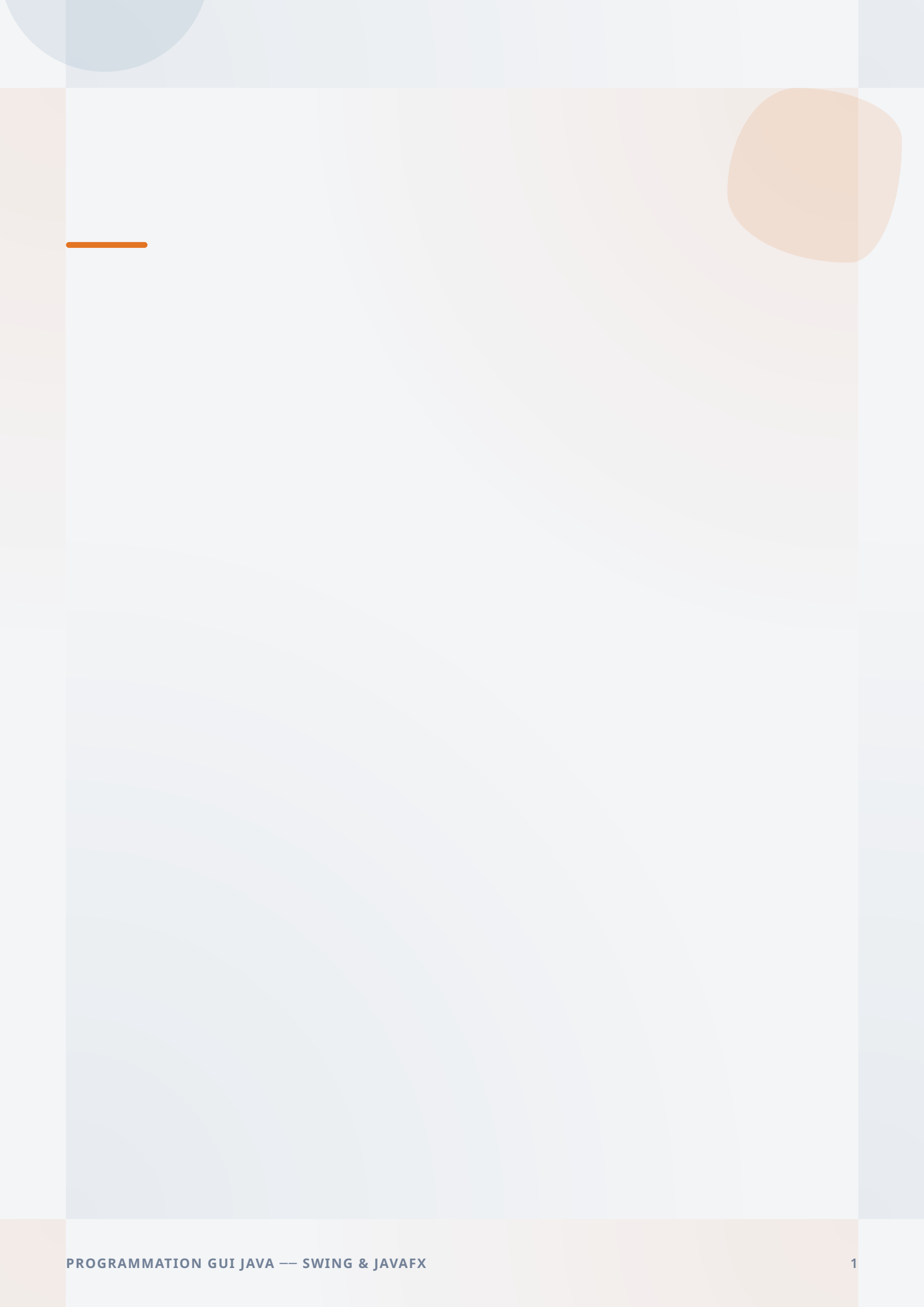




JAVA : INTERFACES GRAPHIQUES (GUI)

Conception et développement d'applications desktop modernes

Un manuel de référence pour maîtriser la programmation événementielle, les gestionnaires de disposition (Layouts), ainsi que les frameworks Swing et JavaFX.

TECHNOLOGIE

Java / Swing & JavaFX

MODULE

Ingénierie Logicielle

NIVEAU

Intermédiaire à Avancé

Sommaire & Feuille de Route

Ce manuel d'apprentissage aborde le développement applicatif desktop en Java sous l'angle de l'architecture découplée, de l'ergonomie et de la programmation réactive.

Chapitre 1 : Introduction et Concepts Fondamentaux des GUI	Page 3
Chapitre 2 : Création de Fenêtres et Composants de base (Swing)	Page 4
Chapitre 3 : Les Layout Managers (Gestionnaires de disposition)	Page 5
Chapitre 4 : La Programmation Événementielle (Listeners)	Page 6
Chapitre 5 : JavaFX - La Modernisation des Interfaces	Page 7
Chapitre 6 : Le Pattern MVC (Modèle-Vue-Contrôleur)	Page 8

PHILOSOPHIE DE CONCEPTION

Le développement d'interfaces graphiques performantes demande une séparation nette entre l'état des données d'un programme (le métier) et l'affichage interactif (la vue) pour maximiser la maintenance.

1. Introduction et Concepts Fondamentaux

Le développement d'applications desktop dotées d'interfaces graphiques (GUI - Graphical User Interface) requiert un paradigme d'exécution particulier différent de la console séquentielle : la **programmation événementielle**.

1.1 FONCTIONNEMENT DU THREAD D'ÉVÉNEMENTS

Contrairement aux programmes en mode console dont le cycle d'exécution s'arrête à la fin de la méthode `main`, une application graphique tourne en boucle continue, à l'écoute d'interactions (clic de souris, frappe de touche au clavier, etc.). En Java (Swing), cette boucle est gérée par un thread dédié unique : l'**Event Dispatch Thread (EDT)**.

RÈGLE D'OR DE L'EDT

Ne jamais exécuter de tâche lourde (calcul complexe, requête réseau ou base de données) directement sur l'EDT. Cela gèle l'interface utilisateur. Ces tâches doivent être déportées sur des threads d'arrière-plan (ex : `SwingWorker`).

1.2 L'ARBRE DES COMPOSANTS

Une interface graphique se modélise sous la forme d'un conteneur principal (la Fenêtre) qui héberge une hiérarchie de conteneurs intermédiaires (Panels) contenant eux-mêmes des widgets graphiques terminaux (boutons, zones de saisie, étiquettes).

CONTENEURS DE HAUT NIVEAU (TOP-LEVEL CONTAINERS)

Ce sont les fenêtres autonomes qui communiquent directement avec le gestionnaire de fenêtres du système d'exploitation. En Swing, il s'agit principalement de `JFrame` et de `JDialog`.

2. Création d'Interfaces de base en Swing

Le framework **Swing**, bien qu'historique, reste la bibliothèque graphique standard nativement intégrée dans le JDK. Sa robustesse repose sur son rendu logiciel indépendant du système d'exploitation hôte.

2.1 STRUCTURE MINIMALE D'UNE FENÊTRE SWING

Voici l'implémentation standard respectant l'initialisation asynchrone sécurisée sur l'Event Dispatch Thread (EDT) :

```
import javax.swing.*;
import java.awt.*;

public class MaFenetre extends JFrame {

    public MaFenetre() {
        super("Application Java GUI");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(400, 300);
        setLocationRelativeTo(null); // Centre la fenêtre

        // Ajout de composants
        JLabel label = new JLabel("Bonjour l'utilisateur !", JLabel.CENTER);
        add(label);
    }

    public static void main(String[] args) {
        // Exécution sécurisée sur l'EDT
        SwingUtilities.invokeLater(() -> {
            new MaFenetre().setVisible(true);
        });
    }
}
```

COMPOSANTS DE BASE INDISPENSABLES

- **JButton** : Bouton d'action standard cliquable.
- **JTextField** : Champ de saisie utilisateur à ligne unique.
- **JCheckBox** & **JRadioButton** : Choix booléens et sélections exclusives.

3. Les Layout Managers (Disposition)

En programmation GUI Java, on ne positionne pas les composants au pixel près à l'aide de coordonnées fixes (x, y). On délègue cette tâche à des **Layout Managers** (gestionnaires de disposition) pour assurer la responsivité lors du redimensionnement de la fenêtre.

3.1 LES LAYOUTS FONDAMENTAUX EN SWING

LAYOUT MANAGER	COMPORTEMENT D'AGENCEMENT	ZONE TYPIQUE D'USAGE
BorderLayout	Divise la zone en 5 régions distinctes (NORTH, SOUTH, EAST, WEST, CENTER).	Idéal pour structurer la disposition globale d'une fenêtre principale.
FlowLayout	Aligne les éléments horizontalement, à la suite, puis passe à la ligne.	Idéal pour regrouper une ligne de boutons de validation en bas de page.
GridLayout	Organise les éléments au sein d'une grille stricte (lignes et colonnes de tailles identiques).	Idéal pour les claviers numériques, les formulaires ou les calculatrices.
BoxLayout	Aligne les composants sur une seule ligne (horizontale) ou une seule colonne (verticale).	Idéal pour assembler des empilements de menus ou de barres d'outils.

ASTUCE DE CONCEPTION

Pour construire une interface graphique complexe, la bonne pratique consiste à imbriquer plusieurs sous-conteneurs (`JPanel`) possédant chacun leur propre **Layout Manager** spécifique.

4. La Programmation Événementielle

La mise en mouvement d'une interface graphique s'appuie sur le design pattern **Observateur / Observé**. Les composants graphiques (boutons, champs) émettent des événements captés par des écouteurs (**Listeners**).

4.1 CAPTURE D'UN CLIC DE BOUTON (ACTION EVENT)

L'exemple de code suivant montre l'évolution de l'écriture des écouteurs en Java, de la classe anonyme classique vers la clarté des **expressions Lambda** (introduites en Java 8) :

```
// Méthode classique historique (Classe Anonyme)
bouton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Action déclenchée !");
    }
});

// Équivalent moderne avec Expression Lambda
bouton.addActionListener(e -> System.out.println("Action déclenchée !"));
```

4.2 LES ADAPTATEURS D'ÉVÉNEMENTS (ADAPTERS)

Certains écouteurs surveillent des événements multiples (ex : **MouseListener** surveille le clic, le survol, l'entrée, la sortie et le relâchement). Pour éviter de devoir implémenter toutes les méthodes vides, Java fournit des classes **Adapter** (ex : **MouseAdapter**) qu'il suffit d'hériter pour redéfinir uniquement la méthode d'intérêt.

ATTENTION AUX THREADS

Toute modification d'un élément d'interface graphique déclenchée au sein d'un Listener s'exécute naturellement sur l'EDT, ce qui est correct et sécurisé pour l'état d'affichage.

5. JavaFX - La Révolution Visuelle

Pour remplacer les composants vieillissants de Swing, Oracle a introduit **JavaFX**, un outil de conception d'interfaces modernes exploitant l'accélération matérielle du GPU.

5.1 LES ATOUTS MAJEURS DE JAVA FX

- **Stylisation CSS** : Possibilité d'habiller et de formater graphiquement les composants applicatifs à l'aide de feuilles de style CSS adaptées (JavaFX CSS).
- **Fichier FXML (Découplage)** : Fichiers XML décrivant la structure de l'interface graphique. Cela sépare le design visuel (réalisé avec l'outil d'édition visuelle *Scene Builder*) du code logique de l'application.
- **Propriétés et Data-Binding** : Permet d'unir dynamiquement les données métiers aux widgets d'interface. Toute modification de valeur se propage automatiquement à l'écran sans code supplémentaire de rafraîchissement.

5.2 STRUCTURE MINIMALE D'UNE APPLICATION JAVA FX

La métaphore de construction de JavaFX s'appuie sur le vocabulaire du théâtre : le **Théâtre (Stage)** héberge une **Scène (Scene)** qui affiche les nœuds du graphe de scène (Scene Graph).

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MonAppFX extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Hello JavaFX !");
        btn.setOnAction(e -> System.out.println("Action"));

        StackPane root = new StackPane();
        root.getChildren().add(btn);

        Scene scene = new Scene(root, 300, 250);
        primaryStage.setTitle("Mon Application FX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```


6. Architecture Applicative et Pattern MVC

Afin d'éviter de créer des classes "monolithes" ingérables de plusieurs milliers de lignes mêlant l'accès à la base de données et l'affichage des boutons, l'implémentation du pattern **Modèle-Vue-Contrôleur (MVC)** est indispensable.

6.1 RÔLE DES TROIS COMPOSANTS DU MVC

1. LE MODÈLE (MODEL)

Contient les données de l'application, l'état métier et les règles de validation. Il ignore totalement l'existence de l'interface graphique (Swing ou JavaFX) pour être hautement testable unitairement.

2. LA VUE (VIEW)

Se charge uniquement de l'affichage des composants à l'écran et de la structuration des Layouts. Elle relaie les actions de l'utilisateur au Contrôleur.

3. LE CONTRÔLEUR (CONTROLLER)

Sert de pivot fonctionnel. Il intercepte les requêtes de la Vue, effectue les actions de mise à jour nécessaires sur le Modèle, puis demande à la Vue de se synchroniser.

SYNTHÈSE FINALE

La maîtrise de la conception des interfaces en Java nécessite de comprendre les threads d'exécution graphiques, d'ordonner proprement les éléments avec des Layouts réactifs et d'architecturer son code avec le modèle MVC. Ces compétences sont le socle de toute application métier desktop solide.

Exemple Pratique 1 : Écran de Connexion Swing

Ce premier cas concret illustre la création d'une boîte de dialogue ou fenêtre de connexion sécurisée (Login Screen) en associant des étiquettes, des champs de saisie, un champ de mot de passe masqué et des boutons d'action.

RENDU VISUEL DE L'INTERFACE



CODE SOURCE SWING ASSOCIÉ

```
// Implémentation du formulaire de connexion
JPanel panel = new JPanel(new GridLayout(3, 2, 10, 10));
JLabel userLabel = new JLabel("Nom d'utilisateur :");
JTextField userField = new JTextField(15);
JLabel passLabel = new JLabel("Mot de passe :");
JPasswordField passField = new JPasswordField(15);

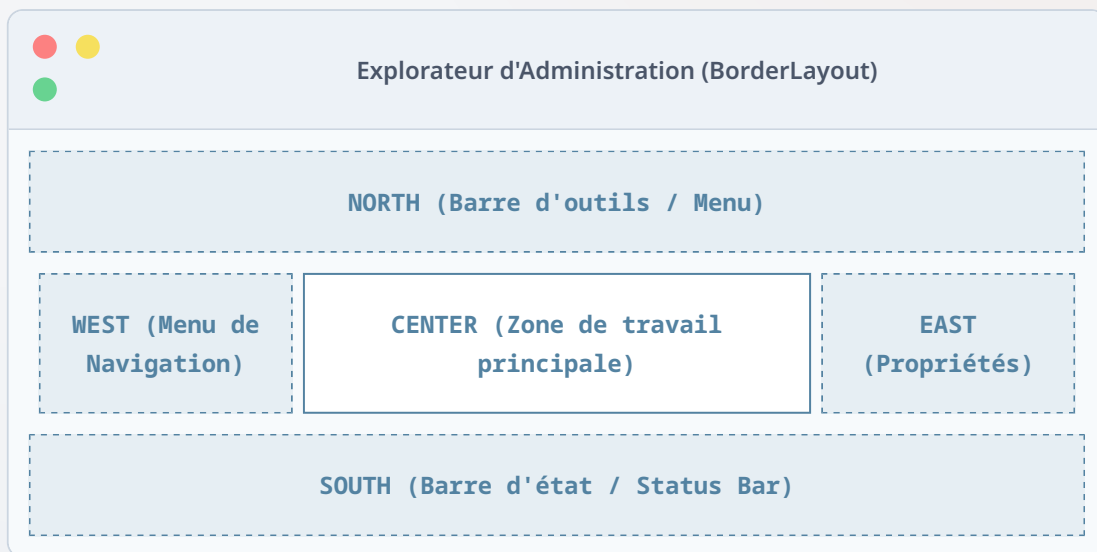
JButton loginButton = new JButton("S'authentifier");
JButton cancelButton = new JButton("Annuler");

panel.add(userLabel);   panel.add(userField);
panel.add(passLabel);   panel.add(passField);
panel.add(loginButton); panel.add(cancelButton);
```

Exemple Pratique 2 : Structure Globale avec BorderLayout

Le BorderLayout est le gestionnaire de disposition par défaut de la classe JFrame. Il est idéal pour construire l'ossature d'une application professionnelle comprenant un menu haut, une barre d'état basse, un volet de navigation latéral et un espace de travail principal.

RENDU SCHÉMATIQUE DE LA DISPOSITION



CODE SOURCE DE L'AGENCEMENT

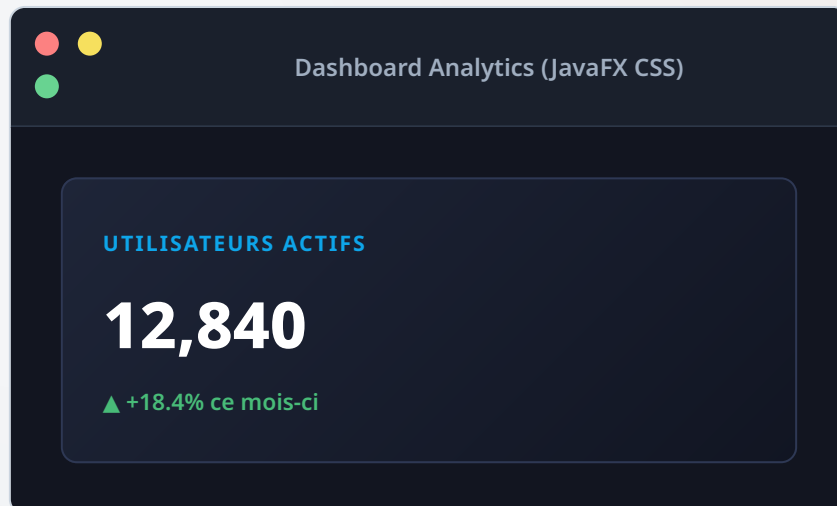
```
// Définition d'un BorderLayout et assignation des zones
frame.setLayout(new BorderLayout(5, 5)); // Espacement h/v de 5px

frame.add(new JButton("NORTH"), BorderLayout.NORTH);
frame.add(new JButton("SOUTH"), BorderLayout.SOUTH);
frame.add(new JButton("EAST"), BorderLayout.EAST);
frame.add(new JButton("WEST"), BorderLayout.WEST);
frame.add(new JPanel(), BorderLayout.CENTER); // Conteneur principal
```

Exemple Pratique 3 : Dashboard Moderne JavaFX & CSS

Contrairement à Swing, JavaFX sépare radicalement la structure visuelle et le design graphique. Cet exemple montre comment concevoir une carte d'indicateur clé de performance (KPI Card) de type Dashboard Web à l'aide de feuilles de style CSS applicatives (fichiers `.css`).

RENDU VISUEL DE LA CARTE GRAPHIQUE



CODE SOURCE FXML ET FICHIER CSS ASSOCIÉ

```
/* Code CSS d'accompagnement (style.css) */
.kpi-card {
    -fx-background-color: linear-gradient(from 0% 0% to 100% 100%, #1e2538, #111522);
    -fx-border-color: #2e3854;
    -fx-border-radius: 8px;
    -fx-padding: 20;
}
.kpi-title {
    -fx-text-fill: #0ea5e9;
    -fx-font-size: 11px;
    -fx-font-weight: bold;
}
.kpi-value {
    -fx-text-fill: #ffffff;
    -fx-font-size: 32px;
    -fx-font-weight: 900;
}
```